SIMATS ENGINEERING

CO3-ASSESSMENT TOOL 1

Experiments

COURSE NAME: Computer Networks **COURSE CODE: CSA07**

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

Code of Conduct:

I A n t o n y M a g r i n (1 9 2 5 7 2 1 0 3) my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student Antony magrin

Criteria	Understanding of Concepts 2.5	Experimental Design 3	Use of Tools 2	Analysis of Results 1.5	Documentation 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

Experiments

QUESTIONS:4 **Total Marks:40**

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, and ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.(BL4) (10)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS

Q1. TCP Three-Way Handshake (SYN, SYN-ACK, ACK)

Procedure:

1. Open Wireshark, start capture on your active interface (Wi-Fi/Ethernet).
2. Apply filter: tcp.flags.syn==1 || tcp.flags.ack==1
3. Open a browser and visit any site (e.g., http://example.com), or run telnet example.com 80 from terminal.
4. Stop capture, filter with: tcp.port==80 && tcp.flags

What you'll see — three packets:

Packet	Source → Dest	Flags	Seq/Ack
1	Client → Server	**SYN** = 1	Seq = x (client's Initial Sequence Number)
2	Server → Client	**SYN, ACK** = 1	Seq = y, Ack = x+1
3	Client → Server	**ACK** = 1	Seq = x+1, Ack = y+1

Role of each flag:

- **SYN (Synchronize):** Sent by the client to initiate a connection and propose its Initial Sequence Number (ISN). It tells the server "I want to start a session, and my byte stream starts at sequence number x."
- **SYN-ACK:** The server acknowledges the client's SYN (Ack = x+1) and simultaneously sends its own SYN with its own ISN (y), proposing its side of the sequence numbering.
- **ACK:** The client acknowledges the server's SYN (Ack = y+1), confirming both sides now agree on starting sequence numbers.

How this establishes reliability:

- Both sides exchange and agree on **Initial Sequence Numbers**, which are used later to detect lost, duplicate, or out-of-order segments.
- The 3-step exchange confirms **bidirectional reachability** — client can reach server, and server can reach client — before any data is sent.
- It prevents old/duplicate connection requests from being interpreted as new ones (protection against stale segments).
- Only after this handshake does TCP move to the ESTABLISHED state, ensuring data transfer only begins once both endpoints have synchronized state.

Q2. DNS over UDP — Header Comparison with TCP

Procedure:

1. Filter: udp.port==53
2. Run nslookup google.com or ping google.com (or clear DNS cache and reload a browser page) to trigger a fresh DNS query.
3. Click on the DNS packet → expand **User Datagram Protocol** in the detail pane.
4. For comparison, click any TCP packet (e.g., from Q1 capture) → expand **Transmission Control Protocol**.

Header size comparison:

Field	UDP Header	TCP Header
Total size	**8 bytes** (fixed)	**20 bytes minimum** (up to 60 with options)
Fields present	Source Port, Dest Port, Length, Checksum	Source/Dest Port, Sequence #, Ack #, Data Offset, Flags, Window Size, Checksum, Urgent Pointer, Options
Sequence numbers	**Absent**	Present
Acknowledgment numbers	**Absent**	Present
Connection state (flags)	**Absent**	Present (SYN/ACK/FIN/RST etc.)

Impact of this lightweight design:

- **Speed:** No handshake is needed before sending data — the DNS query goes out in a single UDP datagram, and the response typically arrives in one datagram too. This makes DNS resolution very fast (often a few milliseconds), which matters since a single webpage load can trigger many DNS lookups.
- **No congestion/flow control overhead:** UDP doesn't perform slow-start or windowing, so there's no delay building up a connection.
- **Reliability trade-off:** Since there are no sequence numbers or acknowledgments, UDP cannot detect lost or out-of-order datagrams at the transport layer. If a DNS query or response is lost, there is no automatic retransmission by UDP itself — the **application (the DNS resolver/client)** must implement its own timeout and retry logic.
- This is an acceptable trade-off for DNS because queries are small, idempotent (asking again causes no harm), and speed matters more than transport-layer reliability guarantees.

Q3. Simulating Packet Loss — TCP Retransmission vs UDP

Procedure:

1. Start a TCP file transfer (e.g., large HTTP download, or FTP) and a UDP stream (e.g., a ping flood or streaming media) simultaneously.
2. While the transfer is in progress, briefly disable Wi-Fi/pull the network cable for 3–5 seconds, then reconnect.
3. Filter TCP traffic: `tcp.analysis.retransmission`
4. Observe the UDP stream (e.g., `udp.port==xxxx`) during the same interval.

What you'll observe:

TCP behavior:

- Wireshark's expert analysis will flag packets as `[TCP Retransmission]` or `[TCP Fast Retransmission]`.
- You'll see the **same sequence number repeated** in multiple packets after the outage.
- The **Retransmission Timeout (RTO)** — visible as a growing gap in packet timestamps — doubles with each failed attempt (exponential backoff).
- After reconnection, you may also see `[TCP Dup ACK]` packets — if the receiver got some packets out of order, it re-sends ACKs for the last in-order byte it received, which can trigger **fast retransmit**.
- Eventually, the connection recovers and continues from where it left off (no data loss to the application).

UDP behavior:

- No retransmission packets at all — datagrams sent during the outage are simply **gone**.
- No error, no retry, no acknowledgment mechanism at the transport layer.
- If you're watching a UDP stream (like audio/video or ping), you'll see a **gap in sequence** (e.g., in RTP payload sequence numbers, if applicable) with nothing filling it in.

Why the difference exists:

- TCP maintains **state** — sequence numbers, ACKs, retransmission timers, and buffers unacknowledged data specifically so it can resend anything not confirmed as received. This guarantees **reliable, ordered delivery** at the cost of added delay (waiting for timeouts, buffering, re-ordering).
- UDP is a **fire-and-forget** protocol: it has no concept of "did this arrive?" It hands the datagram to IP and moves on. This makes it **faster and lower-overhead**, ideal for real-time applications (VoIP, live video, gaming) where a late retransmission is more harmful than a dropped packet — but unsuitable for use cases needing guaranteed delivery unless the application layer adds its own reliability.

In your report, include: a screenshot showing `[TCP Retransmission]` flagged packets with timestamps showing the gap/outage, and (if applicable) a corresponding UDP capture showing no such recovery packets.

Q4. DNS Query Types and Resolution Time

Procedure:

1. Filter: `dns`
2. Clear your OS DNS cache (`ipconfig /flushdns` on Windows, `sudo systemd-resolve --flush-caches` on Linux) so a real query is triggered.
3. In the browser, visit a new domain, or run `nslookup -type=A example.com`, `nslookup -type=AAAA example.com`, `nslookup -type=MX example.com`.
4. Click the query packet, then the matching response packet — Wireshark auto-numbers them so you can find the pair easily (look at **[Response In:]** / **[Request In:]** fields in the DNS detail pane).

What to identify:

- **Query packet:** expand DNS → Queries → note the **Type** field (A, AAAA, MX, etc.) and **Name**.
- **Response packet:** expand DNS → Answers → note the **resource record(s)** returned (IP address for A/AAAA, mail server priority + hostname for MX).

Query Type	Purpose	Example Answer
A	Resolves domain to IPv4 address	93.184.216.34
AAAA	Resolves domain to IPv6 address	2606:2800:220:1:248:1893:25c8:1946
MX	Finds mail server for a domain	Priority + mail server hostname

Measuring resolution time:

- Wireshark shows this directly: in the DNS response packet detail pane, expand **Domain Name System (response)** → look for **[Time: X.XXX seconds]**, which is the delta between query and response.
- Alternatively, subtract the **query packet's timestamp** from the **response packet's timestamp** (visible in the Wireshark packet list "Time" column).
- Typical values are in the range of a few milliseconds (cached at resolver) to 50–150ms (recursive lookup against authoritative servers), depending on network conditions and whether the resolver already had it cached.

Why DNS typically uses UDP:

- DNS queries and responses are almost always **small** (well under the 512-byte traditional UDP payload limit, or under ~4096 bytes with EDNS0), so they fit in a single datagram — no need for TCP's connection setup/teardown overhead.
- **Speed is critical** — DNS lookups happen constantly and transparently before almost every network request, so avoiding the 3-way handshake significantly reduces latency.
- DNS is largely **stateless and idempotent** — if a query is lost, the client's resolver simply retries after a short timeout, which is cheap compared to establishing a TCP connection.
- TCP is used as a **fallback** for DNS when: the response is too large for a single UDP datagram (**truncated**, flagged with the TC bit in the DNS header), for **zone transfers** between DNS servers, or when using DNS-over-TCP/DoT for security reasons.